IL2234

Digital Systems Design and Verification using Hardware Description Languages

Project Milestone 1

ALU and FPGA Implementation

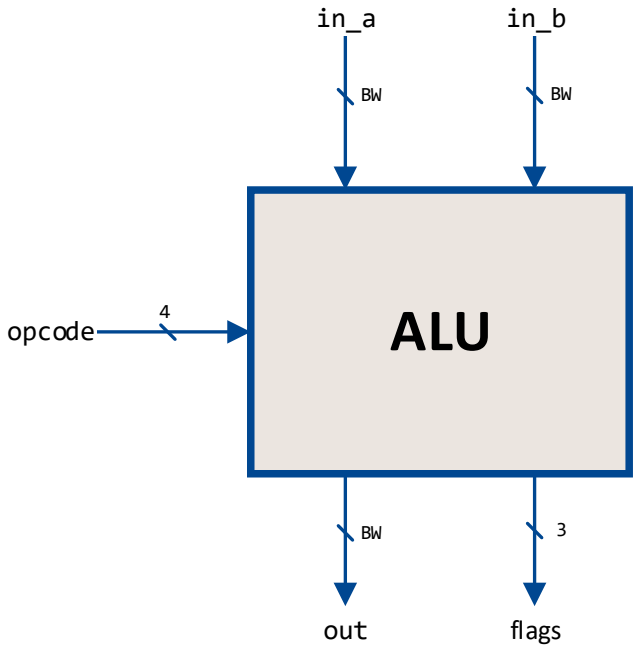
Part 1: Arithmetic Logic Unit (ALU)

Introduction

In this milestone, we will build the first component of our microprocessor: the Arithmetic Logic Unit (ALU). We will design and model the ALU circuit based on the SystemVerilog modelling concepts learned in the class. Additionally, we will develop a simple testbench to test the basic functionality of the ALU.

Overview

Below is a simple diagram of the ALU



The inputs and outputs are described in the table below

Name	Direction	Type	Bitwidth	Description
`in_a`	Input	`logic`	BW [1]	Operand A
`in_b`	Input	`logic`	BW [1]	Operand B
`opcode`	Input	`logic`	4	Operation code
`out`	Output	`logic`	BW [1]	Output result

[1] BW is a configurable parameter that defines the bitwidth of the ALU operands

flags	Output	`logic`	3	Flags of the result (overflow, negative, and zero detect)

The ALU behaviour is defined according to the following table.

Opcode	Name	Output	Description
0000	ADD	in_a + in_b	Addition
0001	SUB	in_a - in_b	Subtraction
0010	SLL	in_a << shamt	Logical left shift
0100	SLT	(in_a < in_b) ? 1 : 0	Set less than (signed)
0110	SLTU	(in_a < in_b) ? 1 : 0	Set less than (unsigned)
1000	XOR	in_a XOR in_b	Logic XOR
1010	SRL	in_a >> shamt	Logical right shift
1011	SRA	in_a >>> shamt	Arithmetic right shift
1100	OR	in_a OR in_b	Logic OR
1110	AND	in_a AND in_b	Logic AND
1111	PASS_B	in_b	Passthrough B

The shift amount is shamt = in_b[$clog2(BW)-1:0]. Operands are interpreted as unsigned in SLTU and SRL, and as 2's complement in SLT, SRA, and the arithmetic operations. For the remaining operations the interpretation does not affect the result. Opcodes not listed in the table give out = 0.

The `flag` output consists of 3 active high flags (overflow, negative, and zero detect) and works as follows:

- **Overflow**: asserted (1) if the result of ADD or SUB gives the incorrect sign, i.e., the operation has an overflow. Deasserted (0) otherwise.
- **Negative**: asserted (1) if the result of **any** operation is a negative number. Deasserted (0) otherwise.

- **Zero detect**: asserted (1) if the result of **any** operation is zero. Deasserted (0) otherwise.

The three flags are combined in a 3-bit output as follows:

```
assign flags = {overflow,negative,zero};
```

Tasks

1. Design the ALU according to the specifications presented earlier. Write your code in System Verilog using **only synthesizable** constructs. Follow the same naming for the signals indicated in the diagrams and tables. The ALU is purely combinational.

2. Write a testbench that generates inputs for the operands and opcodes; you may use randomisation functions to generate these inputs.

3. Simulate the ALU (DUT) in the testbench and check that the results are correct. Generate the necessary stimuli so all operations are checked, with multiple operands for each operation.

Deliverables

1. Submit the RTL design and testbench(es) on your KTH Github repository in *riscv/alu/*

Submission date: *before the examination*

Individual submission: *same content is allowed within a group*

Part 2: FPGA Implementation

Introduction

In this milestone, we will implement the Arithmetic Logic Unit (ALU) from Milestone 1 on the provided FPGA development board. We will use the board's peripherals to showcase and interact with the functionality of the ALU and verify its operation in a practical setting.

Tasks

1. Instantiate the ALU developed in Milestone 1 in the provided top-level module from the lab demo.

2. Use the switches on the FPGA development board to configure and change the two ALU input operands.
3. Display both ALU input operands on the 7-segment displays using the provided display drivers.
4. Provide a way to switch between signed and unsigned representation of the ALU inputs and output.
5. Display the ALU result on the 7-segment displays according to the selected representation.
6. Use the LEDs on the board to display the ALU's ONZ status flags.

Deliverables

No submission required